UART Communication

Introduction to ARM Cortex-M Microprocessors — Chapter 12

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1 Why This Matters

2 Serial Fundamentals

3 Configuring the USART

4 UART as an Instrument

Why This Chapter Matters

So far, seeing what your program does has needed a debugger and a breakpoint. The **UART** changes that. In a few lines, the microcontroller prints text to a terminal on your PC. It is the most useful debugging tool in embedded work. It also links the device to GPS modules, radios and instruments. Use it in every program from now on.

After this chapter, you can:

1. Contrast parallel with serial, and synchronous with asynchronous.
2. Describe every field of a UART frame.
3. Calculate the `BRR` value for a baud rate.
4. Configure a USART and its pins.
5. Send and receive by polling and by interrupt.
6. Use UART output as a diagnostic instrument.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 **Serial Fundamentals**

3 Configuring the USART

4 UART as an Instrument

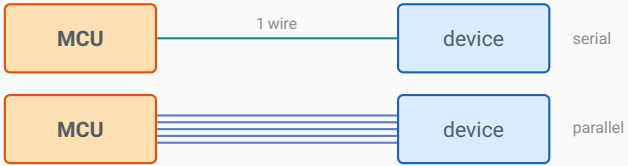
Serial versus Parallel

Serial

- One bit at a time, over a single wire
- Efficient over long distances
- Few pins

Parallel

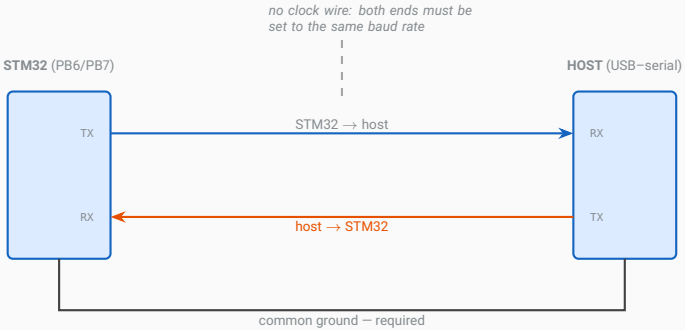
- Many bits at once, over many wires
- Faster over short distances
- Costs pins and board area



Asynchronous means no shared clock — so each byte carries its own framing. **Synchronous** adds a clock wire and removes the need for both ends to agree on speed in advance.

© Muním Zabidi

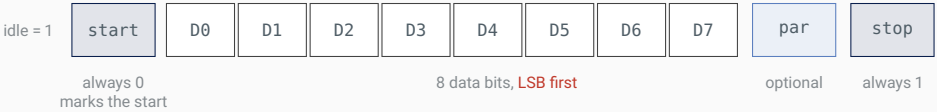
Wiring a UART Link: The Crossover



A UART link is three wires. Each device's TX drives the other's RX — the crossover — and both must share a common ground. There is no clock line, which is why both ends must independently agree on the baud rate.

© Muním Zabidi

The UART Frame



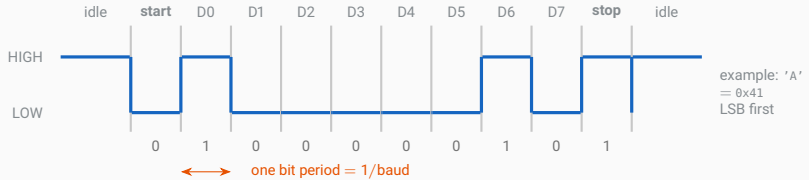
Why asynchronous needs a start bit

With no shared clock, the receiver must know when a byte begins. The line drops from idle-high to low, and the receiver counts bit periods from that edge.

Both ends must agree in advance on baud rate, data bits, parity and stop bits — which is why “9600 8N1” is written down and quoted.

© Munim Zabidi

The Frame, Bit by Bit: Sending `'A'`



The falling edge of the start bit is the receiver's only timing reference. Data bits follow least significant bit first.

UART or USART?

From here on: USART, not UART — that is the name of the actual peripheral on the STM32F446RE.

The U stands for *Universal*

The same hardware can also run synchronous, with an extra clock pin most designs leave unconnected. Used without that pin, a USART behaves exactly like a plain UART — which is how every example here uses it.

Same wire protocol, same frame, same registers below. USART is just the more general peripheral that happens to include UART mode.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 Serial Fundamentals

3 **Configuring the USART**

4 UART as an Instrument

The Registers

Register	Bit	Name	Function
USART_CR1	13	UE	USART enable
	12	M	Word length: 0 = 8 data bits, 1 = 9
	10	PCE	Parity control enable
	5	RXNEIE	Receive-interrupt enable
	3	TE	Transmitter enable
	2	RE	Receiver enable
USART_CR2	13:12	STOP	Stop bits: 00 = 1, 10 = 2 (default: 1)
USART_SR	7	TXE	**T**ransmit register **E**mpty — ready for next byte
	6	TC	Transmission complete — last bit has left the pin
	5	RXNE	**R**eceive register **N**ot **E**mpty — a byte has arrived
USART_DR	7:0 (or 8:0)		Data — write to send, read to receive
USART_BRR	15:0		Baud rate divisor: mantissa in 15:4, fraction in 3:0

On the Nucleo, USART2 is wired to the ST-LINK and appears on your PC as a virtual COM port.

© Muním Zabidi

Baud Rate

$$\text{USARTDIV} = \frac{f_{CK}}{16 \times \text{baud}}$$

The result goes in `BRR`, with the fractional part in the low four bits.

Baud	f_{CK}	BRR
9600	16 MHz	`0x0683`
115200	16 MHz	`0x008B`

Check: $16{,}000{,}000/(16 \times 9600) = 104.17$. The integer 104 is `0x68`, and $0.17 \times 16 \approx 3$ — giving `0x683`.

A mismatched baud rate is the classic first fault

Characters arrive, but as nonsense. If the very first thing you ever receive is garbage, suspect baud rate before anything else.

Configuring USART2

```c
void USART2_init(void) {
  // 1. Clocks: GPIOA on AHB1,
  //    USART2 on APB1
  RCC->AHB1ENR |= (1U << 0);
  RCC->APB1ENR |= (1U << 17);

  // 2. PA2/PA3 to alt. function (10)
  GPIOA->MODER &=  ~((3U<<(2*2)) | (3U<<(3*2)));
  GPIOA->MODER |=  ((2U<<(2*2)) | (2U<<(3*2)));

  // 3. AF7 (USART2), AFR[0]
  GPIOA->AFR[0] &= ~((0xFU<<(2*4)) | (0xFU<<(3*4)));
  GPIOA->AFR[0] |=  ((7U<<(2*4)) | (7U<<(3*4)));

  // 4. Baud: 9600 @ 16 MHz
  USART2->BRR = (104U << 4) | 3U;

  // 5. 8N1, enable TX and RX
  USART2->CR1 = (1U<<3) | (1U<<2);

  // 6. Enable peripheral last
  USART2->CR1 |= (1U << 13);
}
```

Three peripheral blocks, in order: clocks, then pins, then the USART itself. `UE` always comes last — the peripheral ignores configuration changes once it is enabled.

This is the function every program from here on calls once, at the start of `main()`.

TXE and RXNE: The Two Status Bits



Bit	Meaning	Used by
SR bit 7, TXE	Transmit data register empty — safe to write DR	Program 1, every send
SR bit 5, RXNE	Receive data register not empty — a byte is waiting	Program 2, every receive

Same shape, opposite direction

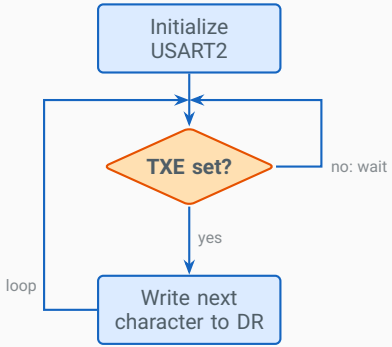
Both programs are the same three-step pattern: initialize, then wait on one status bit, then act in the loop body. TXE waits for room to send; RXNE waits for something to arrive. Everything else in this chapter builds on this shape.

© Muním Zabidi

Program 1: Sending “Hello World!”

```c
void USART2_write(int c)
{
    while (!(USART2->SR
             & (1U << 7))) ;  // wait TXE
    USART2->DR = (c & 0xFF);
}

int main(void)
{
    USART2_init();
    const char *msg = "Hello World!\r\n";
    while (*msg) {
        USART2_write(*msg++);
    }
    while (1) { }
}
```

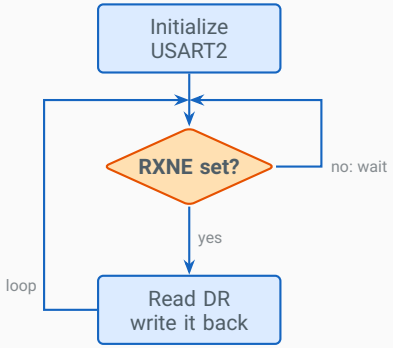


Each character waits for **TXE** (bit 7 of SR) before the next write — one wait loop per byte.

Program 2: Displaying Received Characters

```c
int USART2_read(void)
{
    while (!(USART2->SR
             & (1U << 5))) ;  // wait RXNE
    return USART2->DR;
}

int main(void)
{
    USART2_init();
    while (1) {
        int c = USART2_read();
        USART2_write(c);  // show it
    }
}
```



Reading DR clears **RXNE** (bit 5 of SR) automatically — the hardware never needs to be told explicitly.

Table of Contents

1 Why This Matters

2 Serial Fundamentals

3 Configuring the USART

4 **UART as an Instrument**

The Most Valuable Debugging Tool You Have

What it buys you

- See variables while the program runs, without halting it
- Timestamp events and measure intervals
- Log a fault that happens once an hour

What it costs

At 9600 baud, one character takes about 1 ms. A 40-character line is 40 ms — which can be longer than the deadline you are trying to debug.

The observer effect is real

Adding `printf` to a timing-sensitive routine can make the bug disappear — or create a new one. Use a higher baud rate, shorter messages, or set a GPIO pin and watch it on a scope instead.

Printing Numbers Over UART

```
// Decimal, without printf
void uart_print_uint(uint32_t v)
{
  char buf[11]; int i = 0;
  if (v == 0) {
    uart_write('0'); return;
  }
  while (v > 0) {       // LSD first
    buf[i++] = '0' + (v % 10);
    v /= 10;
  }
  while (i > 0) {       // reverse
    uart_write((uint8_t)buf[--i]);
  }
}

// Hex -- ideal for registers
void uart_print_hex(uint32_t v)
{
  const char *d = "0123456789ABCDEF";
  uart_print("0x");
  for (int s = 28; s >= 0; s -= 4) {
    uart_write((uint8_t)d[(v>>s)&0xF]);
  }
}
```

A few hundred bytes buy a diagnostic channel that works in every program from here on:

```
uart_print("MODER = ");
uart_print_hex(GPIOA->MODER);
uart_print("\r\n");
```

Avoid `printf`: it pulls in tens of kilobytes and may need the heap. These cost almost nothing.

Interrupt-Driven Reception

```
volatile char     rx_buf[64];
volatile uint8_t  rx_head = 0;

void USART2_IRQHandler(void)
{
    if (USART2->SR & (1U << 5)) {        // RXNE
        rx_buf[rx_head++ & 63] = USART2->DR;   // reading DR clears RXNE
    }
}
```

Why polling a UART does not scale

At 115200 baud a byte arrives every 87 µs. If your main loop takes longer than that between polls, the overrun flag sets and data is *lost* — silently.

An interrupt collects each byte the moment it arrives, and `main()` processes the buffer when convenient.

A Second Serial Link: When USART2 Is Not Enough

Every example so far uses USART2, because the Nucleo's built-in ST-LINK makes it the easiest way to talk to a PC. But that link is also its limit: it is already committed. The first time a design needs a second serial device — a GPS module, a Bluetooth radio — while *still* printing debug output, one USART is no longer enough.

```
void USART1_init(void)
{
    RCC->AHB1ENR |= (1U << 1);  // GPIOB
    RCC->APB2ENR |= (1U << 4);  // USART1
                                // -- APB2, not APB1
    // ... same GPIO/AF7 steps,
    // on PB6/PB7 instead of PA2/PA3
    USART1->BRR = 0x0683;
    USART1->CR1 = (1U << 3) | (1U << 2);
    USART1->CR1 |= (1U << 13);
}
```

USART1, on PB6/PB7, is the natural second choice: same AF7 selection, same init shape. The only thing that matters conceptually is that USART1 sits on APB2, while USART2 sits on APB1 — the same f_{CK} question from the baud rate section, now relevant because the two peripherals no longer share a clock source.

Sending One Character at Five Levels

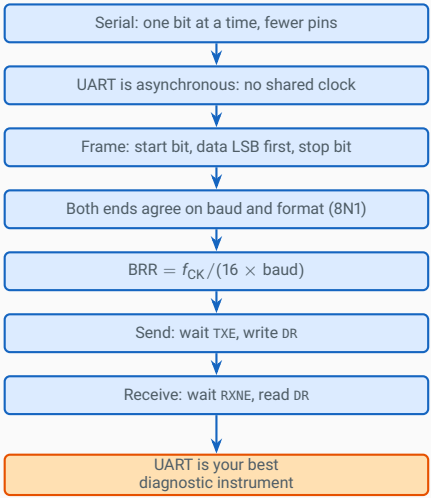
Level	Representation
Physical	PA2 toggles between 0 V and 3.3 V for 10 bit periods
Hardware	Shift register clocked at 9600 Hz; start and stop bits framed
Register	`USART2->DR` written; `TXE` clears, then sets again
C	`uart_write('A');`
Observable	The letter A appears in the PC terminal

UART, from this chapter on

A peripheral you configure and an instrument you reach for in every program. Printing a register value is faster than setting a breakpoint, and the system keeps running while you watch.

© Muním Zabidi

Summary



Terminology covered

- Serial vs parallel
- Start, stop, parity
- Baud rate, BRR
- TXE, RXNE

Next: Chapter 13

SPI and I^2C

A clock wire removes the speed limit.

© Muním Zabidi